

GUIDE UTILISATEUR

Pipeline Kubernetes - configuration, exécution et mesures

Objectif du document

Ce guide explique les opérations nécessaires pour adapter les fichiers de configuration, changer de jeu de données, gérer les nœuds du cluster, lancer le pipeline et démarrer les mesures Ecofloc.

Table des matières

Table des matières.....	1
1. Configuration.....	2
1.1 Fichiers de configuration.....	2
1.2 Jeu de données.....	2
Configuration BERT.....	2
Configuration Embedding.....	2
Script d'initialisation du jeu de données.....	3
Manifestes à ajuster si les chemins internes changent.....	3
1.3 Nœuds.....	4
Format attendu.....	4
2. Exécution.....	4
2.1 Lancement du pipeline.....	4
Catégories de pods.....	5
Clés disponibles.....	5
2.2 Mesures Ecofloc.....	5

1. Configuration

1.1 Fichiers de configuration

Pour utiliser des fichiers de configuration différents de ceux situés dans `code/Energy-Aware-Entity-Resolution/config/examples/`, il faut modifier le script de création des ConfigMaps : `k8s/scripts/configmaps.sh`.

Remplacer le chemin d'accès indiqué à la ligne 42 par le nouveau chemin. Si deux fichiers distincts sont utilisés pour les modes `bert` et `embedding`, et que leur nom contient respectivement ces termes, la variable `CONFIG_TYPE` permet de lancer le pipeline dans l'un ou l'autre mode.

```
41 DECISION_EVALUATION_SCRIPT="${ROOT_DIR}/code/Energy-Aware-Entity-Resolution/distributions/decision_evaluat
42 CONFIG_FILE="${ROOT_DIR}/code/Energy-Aware-Entity-Resolution/config/examples/config-${CONFIG_TYPE}.yaml"
43 KAFKA_PRODUCER_SCRIPT="${ROOT_DIR}/code/Energy-Aware-Entity-Resolution/services/producer.py"
```

Figure 1 - Chemin de configuration utilisé dans `k8s/scripts/configmaps.sh`.

Conseil

Lorsque c'est possible, remplacer les fichiers de configuration d'origine par les nouveaux permet d'éviter de modifier le code du pipeline.

1.2 Jeu de données

Pour changer de jeu de données, plusieurs fichiers doivent être mis à jour afin que les chemins restent cohérents entre la configuration, l'initialisation du PVC et les manifestes Kubernetes.

Élément	Action à réaliser	Lignes
<code>config/examples/</code>	Mettre à jour les chemins du jeu de données pour le mode concerné.	5, 6, 7
<code>k8s/scripts/sync-data-pvc.sh</code>	Indiquer les chemins locaux des fichiers à copier vers le PVC.	15 à 18
<code>k8s/scripts/sync-data-pvc.sh</code>	Optionnel : modifier les chemins de destination utilisés dans les pods.	92 à 96
<code>k8s/pipeline/batch/pipeline.yaml</code>	Si les chemins de destination ont été changés, mettre à jour les paramètres du pipeline.	24, 26, 28
<code>k8s/pipeline/incremental/normalization.yaml</code>	Si nécessaire, modifier le chemin utilisé par le pod <code>normalization</code> .	22

Configuration BERT

```
4 # bert mode, source data
5 trainset_path: "/data/exp_datasets/4-1_dirty_dblp_acm/train.csv"
6 testset_path: "/data/exp_datasets/4-1_dirty_dblp_acm/test.csv"
7 evalset_path: "/data/exp_datasets/4-1_dirty_dblp_acm/valid.csv"
```

Figure 2 - Chemins `trainset`, `testset` et `evalset` pour le mode BERT.

Configuration Embedding

```
4 # incremental mode
5 data_source_A: "/data/exp_datasets/4-1_dirty_dblp_acm/tableA.csv"
6 data_source_B: "/data/exp_datasets/4-1_dirty_dblp_acm/tableB.csv"
7 ground_truth: "/data/exp_datasets/4-1_dirty_dblp_acm/matches.txt"
```

Figure 3 - Chemins `data_source_A`, `data_source_B` et `ground_truth` pour le mode Embedding.

Script d'initialisation du jeu de données

Dans `k8s/scripts/sync-data-pvc.sh`, modifier les chemins locaux pointant vers le dossier du jeu de données et les fichiers nécessaires au pipeline.

```
13 NAMESPACE="${EAER_DATA_NAMESPACE:-argo}"
14 PVC_NAME="${EAER_BERT_DATA_PVC:-pipeline-data-claim}"
15 LOCAL_BERT_DATA_DIR="/srv/shared/data/exp_datasets/4-1_dirty_dblp_acm"
16 GROUND_TRUTH_FILE="/srv/shared/data/exp_datasets/4-1_dirty_dblp_acm/matches.txt"
17 FODORS_TABLE_A_FILE="/srv/shared/data/exp_datasets/4-1_dirty_dblp_acm/tableA.csv"
18 FODORS_TABLE_B_FILE="/srv/shared/data/exp_datasets/4-1_dirty_dblp_acm/tableB.csv"
19 PVC_MANIFEST="${ROOT_DIR}/k8s/pvc-manifests/pvc-data.yaml"
20 SYNC_POD="data-sync-$$"
```

Figure 4 - Variables locales utilisées pour synchroniser le jeu de données vers le PVC.

Optionnel

Les lignes 92 à 96 peuvent être adaptées uniquement si les fichiers doivent être copiés dans un autre chemin à l'intérieur des pods.

```
90 kubectl -n "$NAMESPACE" wait --for=condition=Ready "pod/${SYNC_POD}" --timeout=120s >/dev/null
91 kubectl -n "$NAMESPACE" exec "$SYNC_POD" -- sh -c 'rm -rf /data/*'
92 kubectl -n "$NAMESPACE" exec "$SYNC_POD" -- sh -c 'mkdir -p /data/exp_datasets/4-1_dirty_dblp_acm/'
93 kubectl cp "$LOCAL_BERT_DATA_DIR/. " "$NAMESPACE/$SYNC_POD:/data/exp_datasets/4-1_dirty_dblp_acm"
94 kubectl cp "$GROUND_TRUTH_FILE" "$NAMESPACE/$SYNC_POD:/data/exp_datasets/4-1_dirty_dblp_acm/matches.txt"
95 kubectl cp "$FODORS_TABLE_A_FILE" "$NAMESPACE/$SYNC_POD:/data/exp_datasets/4-1_dirty_dblp_acm/tableA.csv"
96 kubectl cp "$FODORS_TABLE_B_FILE" "$NAMESPACE/$SYNC_POD:/data/exp_datasets/4-1_dirty_dblp_acm/tableB.csv"
```

Figure 5 - Chemins de destination utilisés lors de la copie dans le pod de synchronisation.

Manifestes à ajuster si les chemins internes changent

Dans le manifeste du pipeline `k8s/pipeline/batch/pipeline.yaml`, mettre à jour les paramètres liés aux chemins du jeu de données.

```
19 arguments:
20   parameters:
21     - name: mode
22       value: embedding-evaluation
23     - name: raw_data
24       value: /data/exp_datasets/4-1_dirty_dblp_acm/tableA.csv
25     - name: data_source_A
26       value: /data/exp_datasets/4-1_dirty_dblp_acm/tableA.csv
27     - name: data_source_B
28       value: /data/exp_datasets/4-1_dirty_dblp_acm/tableB.csv
```

Figure 6 - Paramètres `raw_data`, `data_source_A` et `data_source_B` dans le manifeste du pipeline.

Dans le manifeste du pod `normalization` situé dans `k8s/pipeline/incremental/normalization.yaml`, vérifier le chemin passé à la commande de lancement.

```
20 args:
21   - |
22     /opt/venv/bin/python /app/distributions/normalization_distribution.py --mode embedding-inference-inc
```

Figure 7 - Argument de commande du pod `normalization`.

1.3 Nœuds

L'ajout ou la suppression d'un nœud dans ce guide suppose que le nœud a déjà été ajouté au cluster Kubernetes, ou retiré de celui-ci.

1. Lors de l'exécution, le script propose d'assigner les pods à un nœud.
2. Ajouter ou retirer le nœud concerné de la liste des nœuds assignables pour chaque pod.
3. Mettre à jour le script de mesure afin que le nouveau nœud soit pris en compte.

Pour que le script de mesure `k8s/scripts/process.sh` prenne en compte le changement, modifier la liste des nœuds à la ligne 44.

```
44  NODES=(
45      "server2-labo=local:execute"
46      "fedora=ssh:fedora:direct"
47      "server1-k3s-worker=vm:server1:direct"
48  )
```

Figure 8 - Liste des nœuds utilisée par le script de mesure.

Format attendu

`nom_du_nœud=type:[hôte]:mode_ecofloc`

Type	Signification
local	Nœud depuis lequel le script de mesure est exécuté.
vm	Nœud hébergé sur une machine virtuelle.
ssh	Autre nœud accessible depuis la machine locale via SSH.

La valeur `[hôte]` désigne la machine hôte du nœud ou de la machine virtuelle. Utiliser la même notation que pour une commande SSH, par exemple `user@adresse_hôte`. Si le fichier de configuration SSH a été modifié, le nom de l'hôte suffit.

```
> cat ~/.ssh/config
Host server1
    Hostname
    User kevin
```

Figure 9 - Exemple de configuration SSH utilisable pour simplifier le nom de l'hôte.

2. Exécution

2.1 Lancement du pipeline

Pour lancer le pipeline, exécuter la commande suivante : `erctl pipeline start`. Par défaut, le pipeline démarre en mode `Embedding`. Pour lancer le mode BERT, utiliser l'option `-m` : `erctl pipeline start -m bert`.

Déroulement

La commande ouvre d'abord l'éditeur de texte sur le fichier de configuration d'assignation des pods. Après enregistrement et fermeture du fichier, la commande reprend et propose une dernière vérification avant le lancement du pipeline.

```
# Node Scheduling Configuration

batch:
  # Defaults values
  tags: []
  prefer: [server2-labo]
  fallback: [fedora]
  avoid: [server1-k3s-worker, matis-asus-expertbook-b1500ceaey-b1500ceae]

# Bert model tasks
bert:
  tags: [gpu]
  prefer: [server2-labo]
  fallback: [fedora]
  avoid: []
```

Figure 10 - Exemple de configuration d'assignation des pods.

Catégories de pods

La clé `batch` correspond aux pods exécutés par Argo. La clé `incremental` correspond aux pods exécutés directement par Kubernetes.

Les valeurs définies directement sous une catégorie servent de valeurs par défaut. Tous les pods de cette catégorie en héritent. Pour remplacer ces valeurs sur un pod précis, renseigner explicitement un nœud, ou utiliser `null` afin d'écraser la valeur héritée sans renseigner de nœud.

Clés disponibles

Clé	Rôle dans l'assignation des pods
tags	Critère le plus prioritaire. Les nœuds peuvent être étiquetés pour signaler une capacité particulière, par exemple la présence d'un GPU.
prefer	Deuxième critère de priorité. Il permet d'indiquer directement les nœuds à privilégier.
fallback	Critère de dernier recours. Il liste les nœuds à utiliser si les préférences précédentes ne suffisent pas.
avoid	Liste des nœuds à éviter. Les pods ne seront pas assignés à ces nœuds.

2.2 Mesures Ecofloc

Pour que les mesures s'effectuent, le pipeline doit être lancé. L'idéal est de démarrer les mesures avant le pipeline afin de ne pas perdre les premières secondes d'exécution.

Pour lancer les mesures, exécuter : `erctl process ecofloc`.

Selon la configuration des nœuds du cluster, la commande peut demander une ou plusieurs fois le mot de passe sudo, notamment à cause des règles sudoers ou des droits administrateur.

Arrêt et sauvegarde des mesures

La commande fonctionne en mode watch. Pour quitter ce mode et arrêter les mesures, appuyer simultanément sur CTRL + C. Les mesures ne sont pas conservées entre les exécutions : il faut donc les sauvegarder par un moyen externe avant de fermer le terminal.